

Phân tích Scikitlearn và so sánh với hiện thực thủ công

Standard Scaler

Hàm Transform:

```
def transform(self, X, copy=None):
    xp, _, X_device = get_namespace_and_device(X)
    check_is_fitted(self)
    copy = copy if copy is not None else self.copy
    X = validate_data(
        self,
        X,
        reset=False,
        accept_sparse="csr",
        copy=copy,
        dtype=supported_float_dtypes(xp, X_device),
        force_writeable=True,
        ensure_all_finite="allow-nan",
    )
    if sparse.issparse(X):
        if self.with_mean:
            raise ValueError(
                "Cannot center sparse matrices: pass `with_mean=False` "
                "instead. See docstring for motivation and alternatives."
            )
        if self.scale_ is not None:
            inplace_column_scale(X, 1 / self.scale_)
    else:
        if self.with_mean:
            X -= xp.astype(self.mean_, X.dtype)
        if self.with_std:
            X /= xp.astype(self.scale_, X.dtype)
    return X
```

Giải thích:

Tham số copy: Vì python chỉ có truyền Object, không có truyền reference hay truyền value. Nên copy là tham số để quyết định xem phép biến đổi có sửa giá trị được truyền vào không.

```
xp, _, X_device = get_namespace_and_device(X)
```

Kiểm tra xem dữ liệu đến từ thư viện nào, phần cứng đang chạy là cpu hay gpu.

Logic chính :

```
if self.with_mean:
    X -= xp.astype(self.mean_, X.dtype)
if self.with_std:
    X /= xp.astype(self.scale_, X.dtype)
```

Kiểm tra xem kết quả sau khi trừ có còn là ma trận thưa không ? sau đó chuẩn hóa như bình thường.

PCA

```
▼ class PCA(_BasePCA):
    """Principal component analysis (PCA).

    Linear dimensionality reduction using Singular Value Decomposition of the
    data to project it to a lower dimensional space. The input data is centered
    but not scaled for each feature before applying the SVD.

    It uses the LAPACK implementation of the full SVD or a randomized truncated
    SVD by the method of Halko et al. 2009, depending on the shape of the input
    data and the number of components to extract.

    With sparse inputs, the ARPACK implementation of the truncated SVD can be
    used (i.e. through :func:`scipy.sparse.linalg.svds`). Alternatively, one
    may consider :class:`TruncatedSVD` where the data are not centered.

    Notice that this class only supports sparse inputs for some solvers such as
    "arpack" and "covariance_eigh". See :class:`TruncatedSVD` for an
    alternative with sparse data.

    For a usage example, see
    :ref:`sphx_glr_auto_examples_decomposition_plot_pca_iris.py`

    Read more in the :ref:`User Guide <PCA>`.
```

Các solver:

LAPACK:

Tính full SVD nhưng bỏ qua bước tính ma trận hiệp phương sai.

Truncated Randomized SVD với thuật toán Halko.

Thuật toán này cho phép xấp xỉ kết quả với sai số cực kì thấp, nhưng đổi lại là thời gian tính

toán cực kì nhanh. Sử dụng tốt nhất cho ma trận dense

ARPACK:

Tính chính xác K trị riêng, Tối ưu rất tốt cho ma trận thưa.

Cả ARPACK và Halko đều có độ phức tạp thuật toán là

$$O(n \cdot d \cdot k)$$

thay vì implementation thủ công/ Covariance_eigh.

$$O(d^3)$$

Nhưng Class PCA hỗ trợ 3 config chính cho ma trận thưa : auto, arpack, covariance_eigh

```
if issparse(X) and self.svd_solver not in ["auto", "arpack",
"covariance_eigh"]:
    raise TypeError(
        'PCA only support sparse inputs with the "arpack" and'
        f' "covariance_eigh" solvers, while "{self.svd_solver}" was passed.'
        See'
        " TruncatedSVD for a possible alternative."
    )
```

Cách chọn thuật toán ở chế độ auto:

```
if self._fit_svd_solver == "auto":
    # Tall and skinny problems are best handled by precomputing the
    # covariance matrix.
    if X.shape[1] <= 1_000 and X.shape[0] >= 10 * X.shape[1]:
        self._fit_svd_solver = "covariance_eigh"
    # Small problem or n_components == 'mle', just call full PCA
    elif max(X.shape) <= 500 or n_components == "mle":
        self._fit_svd_solver = "full"
    elif 1 <= n_components < 0.8 * min(X.shape):
        self._fit_svd_solver = "randomized"
    # This is also the case of n_components in (0, 1)
    else:
        self._fit_svd_solver = "full"
```

Hàm Covariance_eigh gần với logic của implemnt toán nhất :

```
def _fit_full(self, X, n_components, xp, is_array_api_compliant)
```

Điểm khác biệt chính của fit_full so với implement code toán:

1) Xử lý số âm do lỗi làm tròn

```
eigenvals[eigenvals < 0.0] = 0.0
```

2. Dùng `xp.flip` thay vì `argsort()[::-1]`

3. Tính Singular value để chuyển đổi giữa các solver linh hoạt ví dụ như sang LAPACK :

```
S = xp.sqrt(eigenvals * (n_samples - 1))  
Vt = eigenvecs.T  
U = None
```

vì

$$\lambda = \frac{S^2}{n-1} \implies S = \sqrt{\lambda \times (n-1)}$$

4. Đổi dấu hướng

```
U, Vt = svd_flip(U, Vt, u_based_decision=False)
```

Vì :

$$\cdot (-) = \lambda \cdot (-)$$

và

$$\cdot = \lambda \cdot$$

Đều đúng . Hàm 4) đảm bảo kết quả nhất quán.

LDA:

Đoạn mã này chính là hàm `fit` thực sự của Linear Discriminant Analysis (LDA) trong Scikit-Learn.

Hàm này thực hiện 4 nhiệm vụ chính vô cùng quan trọng: **Validate dữ liệu**, **Tính xác suất tiên định (priors)**, **Khống chế số chiều tối đa**, và **Điều phối solver**.

1. Chuẩn hóa & Validate dữ liệu đầu vào

```
xp, _ = get_namespace(X)
```

```

X, y = validate_data(
    self, X, y, ensure_min_samples=2, dtype=[xp.float64, xp.float32]
)
self.classes_ = unique_labels(y)
n_samples, n_features = X.shape
n_classes = self.classes_.shape[0]

if n_samples == n_classes:
    raise ValueError(
        "The number of samples must be more than the number of classes."
    )

```

- `get_namespace` Giúp code chạy linh hoạt trên cả NumPy, PyTorch, hay CuPy (đây là tính năng mới trong các phiên bản Scikit-Learn gần đây).
- `validate_data`: Đảm bảo n và p hợp lệ, có ít nhất 2 mẫu, thuộc kiểu dữ liệu `float32` hoặc `float64`.
- Ràng buộc n : $n \gg S$ Số mẫu bắt buộc phải nhiều hơn số lớp. Nếu $n = S$, bậc tự do bằng 0 và không thể tính ma trận tán xạ trong lớp (S).

2. Ước lượng Xác suất tiên định (Prior Probabilities (=))

```

if self.priors is None: # estimate priors from sample
    _, cnts = xp.unique_counts(y)
    self.priors_ = xp.astype(cnts, X.dtype) / float(n_samples)
else:
    self.priors_ = xp.asarray(self.priors, dtype=X.dtype)

if xp.any(self.priors_ < 0):
    raise ValueError("priors must be non-negative")

if xp.abs(xp.sum(self.priors_) - 1.0) > 1e-5:
    warnings.warn("The priors do not sum to 1. Renormalizing", UserWarning)
    self.priors_ = self.priors_ / self.priors_.sum()

```

- Nếu không truyền priors: Tự động tính tỉ lệ số mẫu của từng lớp trên tổng số mẫu.
- Kiểm tra hợp lệ: Kiểm tra priors không được âm và tổng các priors phải bằng 1. Nếu tổng không bằng 1, thư viện sẽ tự động chuẩn hóa lại (`self.priors / self.priors.sum()`).

3. Giới hạn số chiều tối đa của LDA ((- 1 d))

```

max_components = min(n_classes - 1, n_features)

if self.n_components is None:
    self._max_components = max_components
else:
    if self.n_components > max_components:
        raise ValueError(
            "n_components cannot be larger than min(n_features, n_classes - 1)."
        )
    self._max_components = self.n_components

```

- Số chiều tối đa mà LDA có thể giảm xuống là $(n - 1)$.

4. Điều phối thuật toán tính toán (Solvers)

```

if self.solver == "svd":
    if self.shrinkage is not None:
        raise NotImplementedError("shrinkage not supported with 'svd' solver.")
    if self.covariance_estimator is not None:
        raise ValueError(...)
    self._solve_svd(X, y)

elif self.solver == "lsqr":
    self._solve_lstsq(
        X, y, shrinkage=self.shrinkage,
        covariance_estimator=self.covariance_estimator
    )

elif self.solver == "eigen":
    self._solve_eigen(
        X, y, shrinkage=self.shrinkage,
        covariance_estimator=self.covariance_estimator
    )

```

Tùy thuộc vào `solver` bạn chọn:

1. Mặc định gọi `self._solve_svd(X, y)`. Cách này làm mịn bằng SVD, không tính SS trực tiếp, rất nhanh và chống lỗi ma trận suy biến.
2. eigen: Gọi `self._solve_eigen(...)`. Cách này tính SS và phân rã trị riêng, hỗ trợ thêm kỹ thuật `shrinkage` để chống suy biến.

3. lsqr: Dùng thuật toán bình phương tối thiểu (Least Squares), kết hợp cực tốt với **shrinkage** cho các bài toán phân loại.

5. Xử lý đặc biệt cho Phân loại Nhị phân (Binary Classification)

```
if size(self.classes_) == 2: # treat binary case as a special case
    coef_ = xp.asarray(self.coef_[1, :] - self.coef_[0, :], dtype=X.dtype)
    self.coef_ = xp.reshape(coef_, (1, -1))
    intercept_ = xp.asarray(
        self.intercept_[1] - self.intercept_[0], dtype=X.dtype
    )
    self.intercept_ = xp.reshape(intercept_, (1,))
```

- Nếu bài toán chỉ có 2 lớp (= 2), thay vì lưu 2 ma trận trọng số cho 2 lớp, LDA sẽ rút gọn lại thành 1 vector hệ số (**coef_**) và 1 hằng số (**intercept_**) duy nhất đại diện cho siêu phẳng phân cách (Decision Boundary):

$$= .$$

- Điều này giúp việc dự đoán nhị phân tương tự như Logistic Regression!

Kết quả:

MNIST_784:

solver covariance_eigh

Accuracy of KNN without dimensionality reduction: 0.943

Accuracy of KNN with PCA: 0.9589047619047619

Accuracy of KNN with manual PCA: 0.9589047619047619

Accuracy of KNN with LDA: 0.9136190476190477

Accuracy of KNN with manual LDA: 0.9126666666666666

Accuracy of KNN with PCA -> LDA: 0.918

Note

PCA của sklearn và PCA cho kết quả KNN hoàn toàn giống nhau. Trong khi LDA thử công lại cho kết quả kém hơn LDA của sklearn.

breast cancer dataset:

solver covariance_eigh

Accuracy of KNN without dimensionality reduction: 0.9590643274853801

Accuracy of KNN with PCA: 0.9649122807017544
Accuracy of KNN with manual PCA: 0.9649122807017544
Accuracy of KNN with LDA: 0.9532163742690059
Accuracy of KNN with manual LDA: 0.9532163742690059
Accuracy of KNN with PCA -> LDA: 0.9824561403508771

Note

cả PCA và LDA đều cho kết quả KNN hoàn toàn giống nhau ở 2 implementation.

Fashion-MNIST

solver covariance_eigh
Accuracy of KNN without dimensionality reduction: 0.853
Accuracy of KNN with PCA: 0.8588095238095238
Accuracy of KNN with manual PCA: 0.8588095238095238
Accuracy of KNN with LDA: 0.8297142857142857
Accuracy of KNN with manual LDA: 0.8313333333333334
Accuracy of KNN with PCA -> LDA: 0.8258571428571428

Title

PCA thủ công và PCA sklearn vẫn tiếp tục giống nhau do solver mà sklearn chọn vẫn là covariance_eigh. Điểm đặc biệt ở đây là LDA thủ công lại cho kết quả vượt trội hơn thuật toán SVD của sklearn.

Isolet

solver randomized
Accuracy of KNN without dimensionality reduction: 0.85
Accuracy of KNN with PCA: 0.8559829059829059
Accuracy of KNN with manual PCA: 0.8538461538461538
Accuracy of KNN with LDA: 0.9478632478632478
Accuracy of KNN with manual LDA: 0.9452991452991453
Accuracy of KNN with PCA -> LDA: 0.938034188034188

Note

Ở dataset này, Sklearn chọn thuật toán randomized, cho kết quả tốt hơn so với covariance_eigh

LDA thủ công có kết quả kém hơn LDA của sklearn một chút.

LFW People

solver randomized

Accuracy of KNN without dimensionality reduction: 0.29437706725468576

Accuracy of KNN with PCA: 0.29988974641675853

Accuracy of KNN with manual PCA: 0.298787210584344

Accuracy of KNN with LDA: 0.13230429988974643

Accuracy of KNN with manual LDA: 0.13230429988974643

Accuracy of KNN with PCA -> LDA: 0.4829106945975744

Note

Kết quả của PCA cũng giống như Isolet, chỉ khác ở LDA có kết quả giống nhau.